

## Servidor de Aplicaciones Bajo Estrés

### 1. DESCRIPCIÓN GENERAL

Este mini proyecto integra los conceptos fundamentales de la administración de sistemas operativos basados en Linux, con énfasis en la gestión de procesos, el monitoreo de recursos y la toma de decisiones ante condiciones de sobrecarga. El estudiante trabaja sobre un entorno real de servidor construido con tecnologías de contenedores Docker sobre Ubuntu WSL2, en el que coexisten tres servicios productivos: un servidor de aplicaciones web (Next.js), un motor de base de datos relacional (PostgreSQL) y un entorno de análisis de datos con Jupyter Notebook.

La práctica está estructurada alrededor de dos fuentes de estrés simultáneas que el propio estudiante diseña e implementa: una aplicación web que inyecta peticiones masivas al servidor y a la base de datos, y un proceso de entrenamiento de un modelo de inteligencia artificial que consume intensivamente CPU y memoria RAM. La concurrencia de ambas cargas genera condiciones de saturación observables y medibles, replicando escenarios reales que ocurren en ambientes de producción.

El valor formativo no reside únicamente en ejecutar herramientas de monitoreo, sino en desarrollar la capacidad analítica para correlacionar una métrica anómala con su proceso causante, comprender el mecanismo físico que produce el colapso y fundamentar una decisión de optimización. Estos son los tres pilares que evalúa el miniproyecto.

### 2. OBJETIVOS

#### 2.1 Objetivo General

Aplicar técnicas de monitoreo y administración de procesos en Linux Ubuntu para identificar, analizar y gestionar situaciones de sobrecarga de recursos en un servidor de aplicaciones multicapa, utilizando herramientas del sistema operativo y tomando decisiones fundamentadas para mejorar el rendimiento.

## 2.2 Objetivos Específicos

| ID  | Objetivo Específico                                                                                                                                                                                                           |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OE1 | Configurar y validar un entorno de contenedores Docker con servicios de servidor web (Next.js), base de datos (PostgreSQL) y entorno de análisis de datos (JupyterLab) sobre Ubuntu WSL2.                                     |
| OE2 | Diseñar e implementar una aplicación web que genere carga controlada y medible sobre los componentes del servidor (CPU, RAM, I/O de disco y red) mediante inyección masiva de peticiones HTTP y consultas a la base de datos. |
| OE3 | Ejecutar simultáneamente la aplicación de estrés web/BD y el notebook de entrenamiento de modelos de IA, induciendo condiciones de saturación de recursos en el sistema operativo.                                            |
| OE4 | Utilizar herramientas de monitoreo del SO (htop, vmstat, iostat, docker stats, pg_stat_activity) para detectar, identificar y analizar los procesos causantes de la sobrecarga.                                               |
| OE5 | Aplicar estrategias de gestión de procesos y recursos para mejorar el rendimiento del servidor bajo condiciones de alta demanda y documentar el impacto de cada medida.                                                       |

## 3. CONTEXTO TÉCNICO DEL ENTORNO

### 3.1 Arquitectura del Servidor

El servidor simulado en este laboratorio replica la arquitectura de un sistema de producción moderno con múltiples capas. Cada capa está aislada en su propio contenedor Docker, pero todos comparten los recursos físicos del sistema operativo anfitrión (Ubuntu WSL2), lo que hace posible observar la contención de recursos cuando múltiples servicios compiten por CPU, memoria y acceso a disco simultáneamente.

- Contenedor 1 — Servidor Web: Next.js 14 con App Router, expuesto en el puerto 3000 o 3001.
- Contenedor 2 — Base de Datos: PostgreSQL 16, con volumen persistente para datos.
- Contenedor 3 — PgAdmin: Panel de administración web para PostgreSQL.
- Contenedor 4 — JupyterLab: Entorno de notebook Python con PyTorch y psutil para el modelo de IA.

### 3.2 Verificación del Entorno

Antes de iniciar las actividades, el estudiante debe confirmar que los contenedores están operativos y verificar los recursos disponibles:



```
$ docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'\n$ docker network ls\n$ free -h && nproc && df -h /
```

Si algún servicio no está activo, levantarlo con:

```
$ docker compose up -d
```

## 4. Actividades del Miniproyecto

### 4.1 Actividad 1 - Diseño e Implementación de la Aplicación Web de Estrés

El grupo debe proponer, diseñar e implementar una aplicación web original que sirva como generador de carga controlada para el servidor. El objetivo técnico es que la aplicación sea capaz de saturar simultáneamente el servidor web (HTTP) y el servidor de base de datos (PostgreSQL) mediante la inyección masiva y configurable de peticiones.

La propuesta es libre en cuanto a la temática de la aplicación (puede ser un simulador de e-commerce, una plataforma de métricas, un gestor de inventario, etc.), pero debe cumplir los requisitos técnicos descritos a continuación. El estudiante puede apoyarse en herramientas de IA generativa para la implementación, empleando los prompts que considere adecuados, pero debe comprender y poder explicar cada componente de lo que entrega. (Debe describir en el informe y generar la evidencia de la funcionalidad de la aplicación, explicando que causa el estrés por el uso de recursos)

#### Requisitos técnicos

- La aplicación debe estar construida en Next.js 14 con App Router y TypeScript.
- Debe conectarse al contenedor PostgreSQL existente usando Prisma ORM o el driver nativo pg.
- Debe incluir al menos tres mecanismos distintos de generación de carga:
  - Inundación de peticiones HTTP concurrentes al servidor web.
  - Inyección masiva de consultas SELECT con JOINS y agregaciones pesadas a PostgreSQL.
  - Inserción en lotes (batch INSERT) para estresar el sistema de escritura y de PostgreSQL.
- Debe contar con un panel de control web (dashboard) desde el cual el estudiante pueda:
  - Iniciar y detener cada mecanismo de estrés individualmente.
  - Configurar parámetros: número de conexiones, duración, tamaño del lote.
  - Visualizar en tiempo real métricas básicas: conexiones activas, CPU%, RAM%. Debe desplegarse como un nuevo servicio en el docker-compose.yml existente, sin alterar los servicios previos.
- El código debe estar documentado y publicado en un repositorio Git con README de instalación.



## Mecanismos de Estrés Mínimos Requeridos

Los siguientes son los mecanismos que deben estar presentes. El grupo puede agregar mecanismos adicionales que considere pertinentes:

| Mecanismo              | Objetivo de Estrés                                                        | Métrica Observable en htop / pg_stat_activity                            |
|------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>HTTP Flood</b>      | Saturar el event loop de Node.js y los workers de Next.js                 | CPU% del proceso node, conexiones en estado 'active' en pg_stat_activity |
| <b>Query Flood</b>     | Maximizar el uso de CPU del proceso postgres mediante consultas complejas | CPU% de postgres, filas en estado 'active' con queries largas            |
| <b>Insert Flood</b>    | Estresar el I/O de disco mediante escritura masiva al WAL de PostgreSQL   | I/O wait en iostat, uso de disco en docker stats                         |
| <b>Lock Contention</b> | Generar bloqueos (locks) entre transacciones concurrentes                 | Estado 'lock wait' en pg_stat_activity, entradas en pg_locks             |

### 4.1 Actividad 2 - Estrés Simultáneo: Aplicación Web + Entrenamiento de Modelo IA

En esta actividad se ejecutan de forma simultánea la aplicación web de estrés (Actividad 1) y el notebook de entrenamiento de un modelo de red neuronal provisto por el docente. La ejecución concurrente de ambas cargas induce un estado de saturación global del sistema operativo en el que los recursos de CPU, RAM y disco son disputados por múltiples procesos al mismo tiempo.

El notebook provisto implementa el entrenamiento de una red neuronal convolucional profunda sobre un dataset de imágenes de alta resolución generadas de forma sintética. El proceso de entrenamiento incluye el paso de backpropagation (cálculo de gradientes), que es el punto de máxima demanda de CPU y memoria.

#### El Notebook de Entrenamiento

El archivo training\_ai\_model.ipynb provisto contiene cinco secciones:

1. Setup y verificación de recursos: detecta CPU, RAM, threads de PyTorch y si el entorno es un contenedor.
2. Dataset de estrés masivo: genera imágenes sintéticas de alta resolución (configurable entre 1500×1500 y 3000×3000 píxeles). Cada imagen ocupa entre 0.03 y 0.1 GB de RAM.
3. Modelo neuronal pesado: define una red convolucional con capas densas de hasta 262 millones de parámetros. La construcción del modelo ocupa entre 1 y 2 GB de RAM.



## Universidad del Valle

### Sede Tuluá

Ingeniería de sistemas

Sistemas operativos

Mini-proyecto

(2026-06-03)

4. Entrenamiento con monitoreo: ejecuta el loop de entrenamiento con backward pass completo, reportando métricas de RAM y CPU en cada batch.
5. Opción adicional de estrés con función específica

**Ajuste según RAM disponible** — Antes de ejecutar el notebook, ajustar el parámetro `IMG_RESIZE`: usar 1500 para equipos con 8 GB de RAM, 2000 para 12 GB y 2500 o superior para 16 GB o más. El parámetro `BATCH_SIZE` debe permanecer en 1 si no hay GPU disponible.

### Procedimiento de Ejecución Simultánea

Seguir este orden para garantizar que ambas cargas estén activas al mismo tiempo:

1. Abrir cuatro terminales WSL independientes y organizarlas en la pantalla (se puede usar Windows Terminal con split panes).
2. Terminal 1 — Monitor principal: ejecutar `htop` y configurar la vista para mostrar el árbol de procesos (tecla F5) y filtrar por docker (tecla F4 → 'docker').
3. Terminal 2 — Docker stats: ejecutar `watch -n1 'docker stats --no-stream'` para ver el consumo de cada contenedor actualizado cada segundo.
4. Terminal 3 — vmstat: ejecutar `vmstat 2 60 > vmstat_estres.txt` para registrar 60 muestras de métricas del sistema durante 2 minutos.
5. Desde el navegador, acceder al dashboard de la aplicación web y activar los mecanismos de estrés (HTTP Flood + Query Flood al mismo tiempo).
6. Acceder a JupyterLab (puerto 8888) y ejecutar las celdas del notebook en orden, sin pausas entre la celda 3 y la celda 4.
7. Observar, analizar y capturar evidencias durante al menos 5 minutos de ejecución concurrente.
8. Detener la aplicación web desde el dashboard. Luego interrumpir el kernel del notebook. Esperar 2 minutos y capturar el estado de recuperación del sistema.

## 5. Monitoreo del Sistema Operativo

### 5.1 Herramientas de Monitoreo

| Herramienta             | Categoría                | Función en el Laboratorio                                                                                                                                                        |
|-------------------------|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>htop</b>             | Monitoreo interactivo    | Visualización en tiempo real de CPU por núcleo, memoria RAM/swap, árbol de procesos, carga promedio y posibilidad de enviar señales (kill, nice) directamente desde la interfaz. |
| <b>vmstat</b>           | Estadísticas de VM       | Reporta bloques de memoria virtual, swap, I/O de bloques, interrupciones del sistema y actividad de CPU en intervalos configurables.                                             |
| <b>iostat</b>           | I/O de dispositivos      | Muestra tasas de lectura/escritura por dispositivo de almacenamiento, útil para detectar cuellos de botella en disco durante escritura masiva a PostgreSQL.                      |
| <b>docker stats</b>     | Métricas de contenedores | Monitoreo en tiempo real de CPU%, RAM usada/límite, I/O de red y disco por contenedor, permite comparar consumo entre servicios.                                                 |
| <b>pg_stat_activity</b> | Estado de PostgreSQL     | Vista del sistema de PostgreSQL que expone todas las conexiones activas, su estado (idle, active, lock wait), duración y texto de la consulta en ejecución.                      |
| <b>stress-ng</b>        | Generación de carga      | Herramienta de referencia para estrés sintético de CPU, memoria, I/O y red; útil como línea base de comparación con la carga generada por la aplicación.                         |
| <b>watch</b>            | Ejecución periódica      | Ejecuta cualquier comando en intervalos regulares mostrando la salida actualizada; ideal para combinar con vmstat o free -h.                                                     |

### 5.2 Comandos de Referencia Rápida

Monitoreo general del SO

```
# htop con árbol de procesos
$ htop -d 5
```

```
# vmstat con muestreo cada 2 segundos
$ vmstat 2 30
```



```
# iostat extendido  
$ iostat -xz 2 10
```

#### Monitoreo de contenedores

```
# Estadísticas en tiempo real  
$ docker stats
```

```
# Procesos dentro de un contenedor específico  
$ docker top nombre_contenedor
```

```
# Logs en tiempo real  
$ docker logs -f nombre_contenedor
```

#### Consultas de diagnóstico en PostgreSQL

Ejecutar estas consultas desde PgAdmin o desde psql mientras las cargas están activas:

```
-- Conexiones activas agrupadas por estado  
SELECT state, count(*) FROM pg_stat_activity GROUP BY state;
```

```
-- Queries de larga duración en ejecución  
SELECT pid, now() - query_start AS duration, state, left(query, 80)  
FROM pg_stat_activity WHERE state != 'idle' ORDER BY duration DESC;
```

```
-- Bloqueos activos  
SELECT pid, relation::regclass, mode, granted  
FROM pg_locks WHERE NOT granted;
```

```
-- Cancelar todas las queries activas (acción de recuperación)  
SELECT pg_cancel_backend(pid) FROM pg_stat_activity  
WHERE state = 'active' AND query_start < now() - interval '30 seconds';
```

## 6. Qué Observar y Cómo Analizarlo

El ejercicio analítico central del miniproyecto consiste en establecer la cadena causal: acción ejecutada → proceso afectado → recurso saturado → métrica observable → decisión de gestión. A continuación se describen los fenómenos esperados en cada escenario:

### 6.1 Durante la Inyección Web + BD

- En htop: el proceso node (servidor Next.js) aparecerá en los primeros lugares por CPU%. El proceso postgres aparecerá con múltiples hilos workers activos.
- En docker stats: el contenedor Next.js mostrará un alto porcentaje de CPU (puede superar el 100% si usa múltiples threads). El contenedor PostgreSQL mostrará incremento en I/O de disco.
- En pg\_stat\_activity: se verán decenas de filas con state='active'. Algunas pueden aparecer en estado 'lock wait' si se activa el mecanismo de contención.
- El load average del sistema (visible en htop, esquina superior derecha) superará el número de núcleos lógicos disponibles, indicando saturación de la cola de procesos.

### 6.2 Durante el Entrenamiento del Modelo IA

- En htop: el proceso python3 (dentro del contenedor Jupyter) consumirá entre el 90% y el 200% de CPU (puede usar múltiples cores con PyTorch).
- La memoria RAM del sistema mostrará un incremento sostenido. Si el modelo excede la RAM disponible, el kernel activará el mecanismo OOM (Out Of Memory Killer) y puede terminar el proceso.
- En vmstat: la columna 'si' y 'so' (swap in/out) pueden mostrar actividad si la RAM se agota, lo que indica degradación severa del rendimiento.

### 6.3 Durante la Ejecución Simultánea

- Este es el escenario de mayor interés. La contención de recursos entre node, postgres y python3 producirá uno o más de los siguientes efectos:
  - Aumento del tiempo de respuesta de la aplicación web (latencia observable en el dashboard).
  - Reducción de la velocidad de entrenamiento del modelo (los batches tardarán más)
  - Incremento del load average por encima del número de CPUs (visible en htop).
  - Posible activación del OOM Killer si la RAM se agota completamente.
  - Activación de swap, visible en vmstat columna 'so' > 0.



## 7. Decisiones de Gestión y Optimización

- Cambiar la prioridad (nice) del proceso de entrenamiento: `renice -n 10 -p <PID>`
- Limitar los CPUs disponibles para el contenedor Jupyter: `docker update --cpus 1.0 nombre_jupyter`
- Reducir el número de workers del DataLoader en el notebook: `num_workers=0` o `num_workers=1`

### 7.2 RAM Agotada

- Reducir `IMG_SIZE` en el notebook (celda 2) antes de reiniciar el entrenamiento.
- Limitar la memoria del contenedor Jupyter: `docker update --memory 3g nombre_jupyter`

## 8. Entregables

| #  | Entregable               | Descripción                                                                                                                                                                                                                                | Peso |
|----|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| E1 | Aplicación web de estrés | Código fuente completo en repositorio Git con README que contenga la descripción de la aplicación, proceso de instalación y ejecución                                                                                                      | 20%  |
| E2 | Video de ejecución       | Grabación de pantalla ( $\geq 10$ min) mostrando ambos escenarios de estrés simultáneo con narración explicativa. youtube.                                                                                                                 | 30%  |
| E3 | Informe técnico IEEE     | Artículo en formato IEEE (mínimo 4 páginas) con abstract, introducción, metodología, resultados, análisis y conclusiones                                                                                                                   | 35%  |
| E4 | Evidencias de monitoreo  | Capturas de pantalla etiquetadas de <code>htop</code> , <code>vmstat</code> , <code>iostat</code> y <code>docker stats</code> en cada escenario, exportadas como PDF (descripción). Incluya todos los archivos <code>.txt</code> generados | 15%  |

### 8.1 Estructura del Video

El video de grabación de pantalla debe mostrar como mínimo las siguientes escenas, narradas con explicación técnica:

1. Presentación de los integrantes del grupo, con descripción del miniproyecto.
2. Estado inicial del sistema: `docker ps`, `htop` en reposo, `docker stats` en baseline.
3. Activación de la Actividad 1 (solo aplicación web): demostración de cada mecanismo de estrés por separado.
4. Análisis en `htop`: identificación del proceso causante, la métrica afectada y la decisión tomada.



## Universidad del Valle

### Sede Tuluá

Ingeniería de sistemas

Sistemas operativos

Mini-proyecto

(2026-06-03)

5. Activación de la Actividad 2 (sólo notebook): ejecución del entrenamiento y observación en htop.
6. Ejecución simultánea: ambas cargas activas. Narrar qué ocurre con CPU, RAM y pg\_stat\_activity.
7. Aplicación de una decisión de optimización y observación del resultado.
8. Estado de recuperación: sistema después de detener ambas cargas.

## 8.2 Estructura del Informe IEEE

El informe debe seguir la plantilla de artículo técnico IEEE con la siguiente estructura mínima:

- Abstract (150–250 palabras): síntesis del trabajo, metodología y resultados principales.
- Introducción: contexto, problema abordado, objetivos y organización del artículo.
- Marco Teórico: gestión de procesos en Linux, monitoreo de recursos, Docker y virtualización ligera.
- Metodología: descripción del entorno, herramientas y procedimiento de ejecución.
- IV. Resultados: datos cuantitativos de CPU%, RAM%, conexiones y latencia para cada escenario. Incluir capturas de htop y tablas comparativas.
- V. Análisis: interpretación de los resultados, identificación de cuellos de botella y justificación de las decisiones tomadas.
- VI. Conclusiones: aprendizajes, limitaciones y trabajo futuro.
- Referencias: mínimo 5 referencias en formato IEEE (libros de SO, documentación oficial de herramientas, artículos).

## 9. Recomendaciones y Buenas Prácticas

### 9.1 Antes de Iniciar

- Verificar que el equipo tenga al menos 8 GB de RAM libre antes de ejecutar la Actividad 2.
- Realizar un snapshot o backup del docker-compose.yml antes de agregar el nuevo servicio.
- Abrir todas las terminales y organizar la pantalla antes de iniciar los escenarios de estrés.
- Ejecutar vmstat 1 5 como prueba de que el comando funciona antes de la sesión principal.

### 9.2 Durante la Práctica

- Capturar pantallas cada vez que se observe un evento relevante (pico de CPU, activación de swap, aparición de lock wait).
- Anotar en un documento temporal los timestamps de cada evento para facilitar el análisis en el informe.

- Si el sistema se vuelve irresponsivo, usar Ctrl+C en la terminal del vmstat para detener la captura y esperar a que el OOM Killer actúe.
- No cerrar JupyterLab desde el navegador sin interrumpir primero el kernel (icono de cuadrado en la barra de herramientas).

### 9.3 Para el Informe

- Los datos cuantitativos son más valiosos que las descripciones narrativas. Incluir tablas con valores numéricos de CPU%, RAM%, tiempo de respuesta y número de conexiones.
- Incluya tablas comparativas del estado de los recursos de hardware en los diferentes momentos.
- Citar las herramientas utilizadas referenciando su documentación oficial o el manual page (man htop, man vmstat).
- El análisis debe responder explícitamente: ¿Qué proceso causó el estrés? ¿Qué recurso fue el cuello de botella? ¿Cómo se detectó? ¿Qué se hizo para solucionarlo?

## 10. Rúbrica de evaluación

| Criterio                                            | Excelente (4.5 – 5.0)                                                                                      | Aceptable (3.5 – 4.4)                                                               | Insuficiente (< 3.5)                                              |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Diseño e implementación de la aplicación web</b> | Aplicación funcional, algoritmos de estrés documentados, README completo y código limpio                   | Aplicación funcional con documentación parcial o código con deudas técnicas menores | Aplicación incompleta o sin documentación, errores no controlados |
| <b>Ejecución simultánea y monitoreo</b>             | Ambos escenarios ejecutados al mismo tiempo, capturas en todos los estados (idle, estrés, recuperación)    | Escenarios ejecutados pero no simultáneamente o capturas incompletas                | Solo un escenario o evidencias insuficientes de monitoreo         |
| <b>Análisis e identificación de causas</b>          | Identifica correctamente proceso causante, métrica afectada y decisión tomada para cada escenario          | Identifica la causa general pero sin precisión en la métrica o la solución aplicada | Descripción superficial sin análisis técnico fundamentado         |
| <b>Informe técnico IEEE</b>                         | Formato correcto, redacción técnica clara, resultados con datos cuantitativos y conclusiones fundamentadas | Estructura IEEE parcial o resultados descriptivos sin datos numéricos               | Sin formato IEEE, redacción no técnica o entrega incompleta       |
| <b>Video de sustentación</b>                        | Narración clara de cada evento, relaciona lo visual con el análisis técnico, duración adecuada             | Video completo pero narración incompleta o desvinculada del análisis                | Video muy corto, sin narración o no muestra los eventos críticos  |